

DESIGN REVIEW · DEEP LEARNING COURSE

Group 10: Real vs AI-Generated Audio

01 · MOTIVATION

Problem & business value

The task

Tell AI-generated tracks apart from human ones, to support copyright and platform-fraud detection.

Why precision matters most

False positive (flagging a real artist) is far costlier than false negative (missing an AI track). We optimize for precision.

Against what baseline?

Platforms mostly rely on uploader self-reporting today: the real baseline is "no detection," not "a worse detector." We report recall too, so nothing slips through unnoticed.

02 · APPROACH

Why this is a deep-learning problem

Task type

Binary classification, from raw audio.

Why not rules?

- Heuristics chase old generator signatures, and drift fast as generators change.
- Hand rules don't transfer across genres or eras.
- A network learns the boundary, and re-learns it as generators change.

Why deep, not shallow ML?

Audio has real spatial and temporal structure: convolution and attention exploit it directly, hand-built features can't.

In short: rules don't adapt, networks do, and audio's structure is exactly what convolution and attention are built to exploit.

03 · DATASETS

Data sources, and why each one

- **SONICS**: 97k songs, 48k real (YouTube) + 49k AI-generated (Suno/Udio). Our only source with both classes labeled, the core train/val set.
- **MusicCaps**: 5.5k real clips with human-written captions. Real half of a matched pair with FakeMusicCaps.
- **FakeMusicCaps**: 27.6k AI tracks regenerated from MusicCaps' captions, across 5 generators (MusicGen, MusicLDM, AudioLDM2, Stable Audio Open, Mustango). Cross-generator generalization test, none of these generators appear in SONICS.
- **GTZAN**: 1,000 real tracks, 10 genres including classical and jazz (often instrumental, no lyrics). Precision stress test, since SONICS' real class is sourced via lyric-matched search and skews toward lyrical/vocal songs, GTZAN checks we don't false-positive on instrumental audio.

> **PRACTICAL NOTE**
> SONICS ships the AI-generated audio directly, but real songs must be downloaded ourselves via each track's YouTube ID: no official API for this, we'll use `yt-dlp`. Expect some removed/unavailable videos and possible rate limits on Colab. FakeMusicCaps' Hugging Face listing is metadata-only too (IDs, labels), the actual audio comes from a separate download script.

04 · DATA PIPELINE

Data loader design

SONICS (real+AI) → 10-sec clips → 80/20 train/val

`data_loader.py`

- `_normalize_sample_rate()`: 16kHz mono, one frequency across eras.
- `_apply_augmentation()`: noise + bitrate variation, decouples fidelity from origin.
- `_generate_spectrogram()`: waveform → 2D Mel-spectrogram.

Held-out test (external, never trained on): MusicCaps+FakeMusicCaps: 5 generators, ~500–750 clips, cross-generator generalization. GTZAN: real-only precision stress test.

Baseline: hand-crafted features + logistic regression, as a sanity check only.

05 · ARCHITECTURE A

Mel-CNN: spatial texture

`models.py`

`class SpectrogramCNN(nn.Module)` treats the spectrogram as an image and reads local frequency texture.

- **Layers:** sequential 2D conv blocks, pooling, a small fully-connected head, or a pretrained ResNet backbone via transfer learning.
- **Transfer learning: PANNs (CNN14)**, pretrained on AudioSet, audio-native features, not ImageNet. Freeze backbone, fine-tune the head.
- **Data flow:** 2D spectrogram matrices through `forward(self, x)`.
- **Inductive bias:** local spectral texture: good at catching short, localized synthesis artifacts.

06 · ARCHITECTURE B

Sequential model: temporal structure

`models.py`

`class AudioSequenceModel(nn.Module)` reads audio as a time series rather than an image.

- **Layers:** recurrent blocks (LSTM/GRU) or a Transformer encoder over time-series slices.
- **Transfer learning: MERT**, pretrained on music, encodes rhythm/pitch/structure, reinforcing our temporal-artifact hypothesis. Freeze backbone, fine-tune the head.
- **Data flow:** linear sequences through the forward pass.
- **Inductive bias:** long-range temporal structure: composition, rhythmic intervals, melodic pacing.

07 · HYPOTHESIS

What we expect, and how we'll test it

OUR HYPOTHESIS

The sequential model wins: AI artifacts show up more in rhythm/phrasing than spectral texture.

	MEL-CNN	SEQUENTIAL
Bias	Spectral texture	Temporal structure
Strength	Fast; encoding artifacts	Rhythmic artifacts, our edge
Compute	Lower	Higher

Test: matched compute budget, same held-out set, error analysis checks if errors match the hypothesis.

08 · TRAINING

Training pipeline

`train.py`

TECHNIQUE	WHY
Dropout	Reduces overfitting on limited data.
Batch norm	Stabilizes training.
Fine-tuning backbone	PANNs (CNN14) for Mel-CNN, MERT for sequential, frozen, small head fine-tuned. Fits Colab's compute budget.
Attention	Weighs distant timesteps, matches our hypothesis.

Tooling: W&B (experiment tracking), Optuna (auto hyperparameter search).

09 · EVALUATION

Evaluation & definition of done

`evaluate.py`

- `calculate_business_metrics()`: heavy false-positive penalty.
- `deep_error_analysis()`: errors by decade and genre.
- Multiple seeds, confidence intervals, not one run.
- **DoD:** overfit 10 samples to confirm the pipeline works, then a full run on our subsample (30–50%), not the full 4,751-hour dataset.

Output is a probability (e.g. 92% AI), not just a label, the precision-first decision from slide 1 means picking a stricter threshold, not just tuning the loss.

10 · RISKS

Risks & mitigations

Dataset coverage
Limited genres/eras. **Mitigation:** document gaps.

Generator overfitting
May learn one generator's fingerprint. **Mitigation:** multiple sources in train and test.

Compute budget
Two architectures + sweeps is a lot. **Mitigation:** deeper tuning pass for the sequential model.